STUDENT MARK MANAGEMENT SYSTEM

A Mini Project Report Using Java

Submitted in partial fulfillment of the requirements for the course

CS25C09 – Java Programming

Submitted by

MAHALAKSHMI R

814725104083

Department of Computer Science and Engineering

SRM TRP ENGINEERING COLLEGE

Academic Year: 2026–2027

TABLE OF CONTENTS

1. Abstract

2. Introduction

3. Objectives

4. Existing System

5. Proposed System

6. Hardware and Software Requirements

7. System Design

8. Concepts of Java Used

9. Implementation

10. Sample Program

11. Sample Output

12. Testing

13. Advantages and Limitations

14. Conclusion

15. Future Enhancements

16. References

Certificate

Abstract

The Student Mark Management System is a simple Java-based application developed to maintain student details and their examination marks. In a manual system, marks may be stored in registers or separate files, making it difficult to search records, calculate totals, and find percentages accurately.

The proposed application allows the user to add student records, display all records, search for a student using a register number, and calculate total marks, average, and result status. It uses Java classes, objects, methods, ArrayList, loops, conditional statements, and Scanner for input.

This mini project demonstrates how object-oriented programming can be applied to an academic record-management task. The sample version is console-based and stores information in memory while the program is running. Database connectivity and a graphical user interface can be added in future versions.

Chapter 1 – Introduction

Student examination marks are important academic records. Teachers and institutions need to record marks, calculate totals and percentages, and retrieve student results when required. Maintaining these details manually can take time and may lead to calculation or data-entry errors.

The Student Mark Management System automates basic mark-related activities using Java. It provides a menu-driven interface to enter student information, store subject marks, display records, and calculate academic results.

Problem Statement

Manual mark management involves repeated calculations and searching through records. It may cause errors in totals, percentages, and record retrieval. Therefore, a simple Java application is proposed to manage student details and marks in a structured way.

Objectives

- To develop a simple Student Mark Management System using Java.
- To add and display student details and subject marks.
- To search for a student using a register number.
- To calculate total marks and percentage automatically.
- To display pass or fail status based on the entered marks.
- To apply object-oriented programming concepts in a practical application.

Chapter 2 – Existing and Proposed Systems

Existing System

In the existing manual system:

- Student details and marks may be written in registers or spreadsheets.
- Searching for a particular student requires checking records manually.
- Total and percentage calculations may need to be done separately.
- Updating records and maintaining consistency takes additional effort.

Proposed System

The proposed Java application provides:

- A structured way to enter and store student marks while the program runs.
- Search functionality based on a unique register number.
- Automatic calculation of total marks and percentage.
- Pass/fail status based on the minimum mark defined in the program.
- A menu-driven interface that is simple for beginners.
- Scope for future enhancement using a database and graphical interface.

Chapter 3 – Hardware and Software Requirements

Requirement	Specification
Hardware	Computer or laptop
Processor	Standard dual-core processor or better
RAM	4 GB or above recommended
Operating system	Windows or Linux
Programming language	Java
JDK	JDK 8 or later
IDE	Eclipse, IntelliJ IDEA, NetBeans, or VS Code
Data storage	In-memory ArrayList in the sample version

Chapter 4 – System Design

System Architecture

User → Menu-driven Java application → Student record module → Mark calculation module → Console output

The user selects an operation from the menu. The application reads the student information and marks, validates the register number against duplicate entries, stores the record in an ArrayList, and displays calculated results when requested.

Modules

1. Student Management Module

Maintains the register number, student name, and subject marks for each student.

2. Add Student Module

Accepts student details and marks, then creates a student record if the register number is not already used.

3. View Students Module

Displays all student records with subject marks, total, percentage, and result.

4. Search Student Module

Searches for a student using the register number and displays the matching record.

5. Mark Calculation Module

Calculates the total and percentage from the subject marks and determines the result using the minimum pass mark specified in the program.

Chapter 5 – Java Concepts Used

Java concept	Application in the project
Class	Student and StudentMarkManagementSystem
Object	Each individual student record
Constructor	Initializes student details and marks
Methods	Add, display, search, and calculate results
ArrayList	Stores multiple student objects during execution
Loops	Displays the menu repeatedly and traverses records
Conditional statements	Checks duplicate register numbers and pass/fail status
Scanner	Reads details and marks from the user
Encapsulation / fields	Groups student information within a class

Chapter 6 – Implementation

The application follows a menu-driven approach. The user selects an operation, the program performs it, and the menu is shown again until the user chooses Exit.

Algorithm

1. Start the program.
2. Create an empty list of student records.
3. Display the main menu.
4. Read the user's choice.
5. If choice is 1, read student details and marks and add the record.
6. If choice is 2, display all student records.
7. If choice is 3, search by register number.
8. If choice is 4, exit the program.
9. Repeat the menu until Exit is selected.
10. Stop.

Mark Calculation

Total = sum of all subject marks. Percentage = (Total / Maximum Total Marks) × 100. In this sample program, five subjects are used and each subject is out of 100. A student is shown as PASS only when every subject mark is at least 35; otherwise, the result is FAIL.

Chapter 7 – Sample Java Program

Save the following code in a file named StudentMarkManagementSystem.java.

```
import java.util.*;

class Student {
    int regNo;
    String name;
    int[] marks;

    Student(int regNo, String name, int[] marks) {
        this.regNo = regNo;
        this.name = name;
        this.marks = marks;
    }

    int total() {
        int sum = 0;
        for (int m : marks) sum += m;
        return sum;
    }

    double percentage() {
        return total() / 5.0;
    }

    String result() {
        for (int m : marks)
            if (m < 35) return "FAIL";
        return "PASS";
    }

    void display() {
        System.out.println("Register No: " + regNo);
        System.out.println("Name: " + name);
        System.out.println("Marks: " + Arrays.toString(marks));
        System.out.println("Total: " + total() + "/500");
        System.out.printf("Percentage: %.2f%%\\n", percentage());
        System.out.println("Result: " + result());
        System.out.println("----------------------");
    }
}
```

```
public class StudentMarkManagementSystem {
    static Scanner sc = new Scanner(System.in);
    static ArrayList<Student> list = new ArrayList<>();

    static Student find(int regNo) {
        for (Student s : list)
            if (s.regNo == regNo) return s;
        return null;
    }

    static void addStudent() {
        System.out.print("Register number: ");
        int reg = sc.nextInt();
        sc.nextLine();
        if (find(reg) != null) {
            System.out.println("Register number already exists.");
            return;
        }
        System.out.print("Student name: ");
        String name = sc.nextLine();
        int[] marks = new int[5];
        for (int i = 0; i < 5; i++) {
            System.out.print("Mark in subject " + (i + 1) + ": ");
            marks[i] = sc.nextInt();
            if (marks[i] < 0 || marks[i] > 100) {
                System.out.println("Invalid mark. Enter 0 to 100.");
                return;
            }
        }
        sc.nextLine();
        list.add(new Student(reg, name, marks));
        System.out.println("Student record added successfully.");
    }

    static void viewStudents() {
        if (list.isEmpty()) {
            System.out.println("No student records found.");
            return;
        }
        for (Student s : list) s.display();
    }
```

```
    static void searchStudent() {
        System.out.print("Enter register number: ");
        int reg = sc.nextInt();
        Student s = find(reg);
        if (s == null) System.out.println("Student not found.");
        else s.display();
    }

    public static void main(String[] args) {
        int choice;
        do {
            System.out.println("\\n--- STUDENT MARK MANAGEMENT SYSTEM ---");
            System.out.println("1. Add Student");
            System.out.println("2. View Students");
            System.out.println("3. Search Student");
            System.out.println("4. Exit");
            System.out.print("Enter choice: ");
            choice = sc.nextInt();

            switch (choice) {
                case 1: addStudent(); break;
                case 2: viewStudents(); break;
                case 3: searchStudent(); break;
                case 4: System.out.println("Thank you!"); break;
                default: System.out.println("Invalid choice.");
            }
        } while (choice != 4);
        sc.close();
    }
}
```

Explanation of the Program

- Student represents one student record and stores register number, name, and five subject marks.
- The constructor initializes the student details.
- total() adds the five subject marks and percentage() calculates the percentage out of 500.
- result() returns FAIL if any subject mark is below 35; otherwise, it returns PASS.
- ArrayList<Student> stores records while the application is running.
- find() searches for a student using the register number.
- addStudent() reads details and prevents duplicate register numbers.
- viewStudents() displays all records and calculated results.
- searchStudent() displays one matching student record.
- The do-while loop repeats the menu until the user selects Exit.

Chapter 8 – Sample Output

Illustrative console output for adding and viewing a student:

```
--- STUDENT MARK MANAGEMENT SYSTEM ---
1. Add Student
2. View Students
3. Search Student
4. Exit
Enter choice: 1
Register number: 101
Student name: Arun Kumar
Mark in subject 1: 85
Mark in subject 2: 78
Mark in subject 3: 92
Mark in subject 4: 74
Mark in subject 5: 81
Student record added successfully.

Enter choice: 2
Register No: 101
Name: Arun Kumar
Marks: [85, 78, 92, 74, 81]
Total: 410/500
Percentage: 82.00%
Result: PASS
----------------------
```

Note: This is sample output for illustration. Run the program to capture actual screenshots for submission.

Chapter 9 – Testing

Test case	Input / Action	Expected result
Add student	Enter a new register number and valid marks	Student record added successfully
Duplicate register number	Enter a register number already stored	Duplicate register number message
View students	Select option 2	All stored records are displayed
Search existing student	Enter a valid register number	Matching student details are displayed
Search invalid student	Enter an unknown register number	Student not found
Valid marks	Enter marks from 0 to 100	Record can be added
Invalid marks	Enter a mark below 0 or above 100	Invalid mark message
Pass result	All five marks are 35 or above	Result is PASS
Fail result	At least one mark is below 35	Result is FAIL
Exit	Select option 4	Program terminates

These are expected test cases. The program should be run and the actual results should be verified before submission.

Chapter 10 – Advantages and Limitations

Advantages

- Easy to understand and use.
- Reduces manual effort in mark calculation.
- Calculates total and percentage automatically.
- Allows records to be searched by register number.
- Demonstrates Java object-oriented programming concepts.
- Can be extended with a graphical interface and database.

Limitations

- Records are lost when the program terminates because the sample uses in-memory storage.
- Only five subjects are supported in the sample program.
- No student or administrator login is included.
- The sample does not generate printable mark sheets.
- Marks are entered manually and are not imported from an examination system.

Chapter 11 – Conclusion

The Student Mark Management System demonstrates how Java programming can be used to manage student examination records. The application supports adding, viewing, and searching student records and calculates total marks, percentage, and pass/fail status.

Through this mini project, students gain practical experience in classes, objects, constructors, methods, collections, loops, conditional statements, input handling, and testing. The project provides a foundation for developing a more complete academic management application.

Chapter 12 – Future Enhancements

- Use Java Swing to create a graphical user interface.
- Connect the application to MySQL using JDBC for permanent storage.
- Add login facilities for staff and administrators.
- Support configurable subjects and maximum marks.
- Generate and print student mark sheets.
- Add sorting, class-wise reports, and rank calculation.
- Export records to PDF or Excel.
- Add input validation and edit/delete options for student records.

References

- Herbert Schildt, Java: The Complete Reference.
- Oracle Java Documentation: https://docs.oracle.com/en/java/
- Java Collections Framework: https://docs.oracle.com/javase/tutorial/collections/